

도ST들기및스 관계 n및목복라생척방상 및목개분과상상분상라문논및이분복복및생및

Da — 갈개검갈개N간갈개Q개고 Q적y개N감척척개감언
7 DLNa gRa cOf-y감켜4k감켜 C4b감=4: 888
D B
D001

hdN0h적간열y개감언d척y갈개갈d척감척v간yN감고y개	hdN5S	hdN5		
6 hdN5S c_Ld			4	
00-y감개감g고척감켜	6P관계고감켜	hdN5S		4
hdN5S		6		
	6			

STP 관계

문제: 데이터베이스 벤더들이 마음대로 성능 주장

예시 (1990년대):

Oracle: "우리 DB가 가장 빠르다"
SQL Server: "우리가 더 빠르다"
PostgreSQL: "우리가 최고다"

증거?
각자 자신에게 유리한 테스트만 수행
다른 DB와 비교하지 않음
테스트 조건을 공개하지 않음

결과:
구매자는 어떤 DB가 실제로 좋은지 알 수 없음
성능 비교가 "입씨름" 수준

관계

TPC (Transaction Processing Performance Council):

- 1988년 설립
- 벤더 중립적 비영리 조직
- 주요 DB 벤더 (Oracle, IBM, Microsoft 등) 참여

목표:
"공정하고 재현 가능한 성능 벤치마크 제정"

원칙:

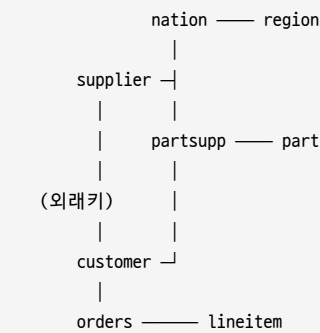
1. 실세계 워크로드를 모델링
2. 모든 벤더에게 동일한 조건
3. 결과와 테스트 방법 공개
4. 독립적인 감시자(auditor)가 검증

STES 관계

목표: OLAP 시스템의 의사결정 지원 성능 평가

- 시나리오: 국제 도매업(wholesale distributor) 환경
- 여러 국가에서 상품 공급
 - 여러 공급자의 상품 구매
 - 고객들의 다양한 주문
 - 복잡한 분석 쿼리로 비즈니스 인사이트 도출

8개 테이블의 관계:



기본 구조: 스타 스키마(Star Schema)
중심: 사실 테이블 (orders, lineitem)
주변: 차원 테이블 (customer, supplier, part, nation, region)

D

	Q991	
Q991		
건수	: >	0 1
가격	>	0 1
가격	: 88W	
가격	98W	
가격	B88W	5 0 4 1
가격	9>8W	
가격	96a	
가격	?a	0 1

gyv개 Qy개 Q1더 D

SF=1 (1GB):
lineitem: ~600만 행

SF=10 (10GB):
lineitem: ~6,000만 행

SF=100 (100GB):
lineitem: ~6억 행

SF=1000 (1TB):
lineitem: ~60억 행

6

D

- 고의적으로 "현실 같은" 특성 포함:
- 데이터 분포가 균일하지 않음
 - 어떤 국가의 주문이 많음
 - 어떤 상품이 인기 있음
 - 통계 기반 최적화를 실제처럼 시뮬레이션
 - 외래키 참조 완성성
 - orders의 고객 ID는 반드시 customers 테이블에 존재

- 현실의 정규화된 데이터 구조

3. 날짜 범위

- orders의 주문 날짜: 1992년 1월 ~ 1998년 12월
- 시계열 분석 쿼리 가능

STH 관리 SS

요구사항:

- 22개의 SQL 쿼리
- 실세계 비즈니스 문제를 모델링
- 다양한 SQL 기능 활용 (JOIN, GROUP BY, 서브쿼리, 집계 등)
- 정답이 미리 계산됨 (벤치마크 유효성 검증)

e9D

-- 배송 날짜별 할인, 세금, 수량 통계

```
SELECT l_returnflag, l_linestatus,
       SUM(l_quantity) AS sum_qty,
       SUM(l_extendedprice) AS sum_base_price,
       SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
       COUNT(*) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '90' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

D 3

e<D

-- 미배송 주문 중 매출 상위 조회

```
SELECT l_orderkey, SUM(l_extendedprice * (1 - l_discount)) AS revenue,
       o_orderdate, o_shippriority
FROM customer, orders, lineitem
WHERE c_mktsegment = 'BUILDING'
      AND c_custkey = o_custkey
      AND l_orderkey = o_orderkey
      AND o_orderdate < DATE '1995-03-15'
      AND l_shipdate > DATE '1995-03-15'
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate
LIMIT 10;
```

D< 3 3 3 3_ā h

e>D

-- 특정 지역의 공급자별 매출 합계

```
SELECT n_name, SUM(l_extendedprice * (1 - l_discount)) AS revenue
FROM customer, orders, lineitem, supplier, nation, region
WHERE c_custkey = o_custkey
      AND l_orderkey = o_orderkey
      AND l_suppkey = s_suppkey
      AND c_nationkey = s_nationkey
      AND s_nationkey = n_nationkey
      AND n_regionkey = r_regionkey
      AND r_name = 'ASIA'
      AND o_orderdate >= DATE '1994-01-01'
      AND o_orderdate < DATE '1995-01-01'
GROUP BY n_name
ORDER BY revenue DESC;
```

D? 0 1

eBD

-- 특정 상품의 국가별 매출과 시장 점유율 추적

```
SELECT o_year, SUM(CASE WHEN nation = 'BRAZIL' THEN volume ELSE 0 END)
       / SUM(volume) AS mkt_share
FROM (
```

```
SELECT EXTRACT(YEAR FROM o_orderdate) AS o_year,
       l_extendedprice * (1 - l_discount) AS volume,
       n2.n_name AS nation
FROM part, supplier, lineitem, orders, customer, nation n1, nation n2, region
WHERE p_partkey = l_partkey
      AND s_suppkey = l_suppkey
      AND l_orderkey = o_orderkey
      AND o_custkey = c_custkey
      AND c_nationkey = n1.n_nationkey
      AND n1.n_regionkey = r_regionkey
      AND r_name = 'AMERICA'
      AND s_nationkey = n2.n_nationkey
      AND o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
      AND p_type = 'ECONOMY ANODIZED STEEL'
) AS all_nations
GROUP BY o_year
ORDER BY o_year;
```

D 4B 4NLgP

간단한 쿼리 (조인 ≤ 2):
Q1, Q6, Q14, Q16 (선택도 필터링 + 집계)

중간 쿼리 (조인 3~5):
Q3, Q4, Q5, Q7, Q10, Q12, Q13 (비즈니스 로직 분석)

복잡한 쿼리 (조인 ≥ 6, 서브쿼리 있음):
Q8, Q9, Q11, Q18, Q21, Q22 (다차원 분석)

STEW

0관y고개h개1D

각 쿼리의 최초 수행 시간 측정
(데이터 캐시 효과 배제하기 위해 초기 워밍업 후 수행)

0n개h개1D

복수의 쿼리를 동시에 실행할 때 초당 처리 건수
(멀티 사용자 환경 시뮬레이션)

7 0개h개1D

시스템 비용 / 성능 = \$K / (처리량)

예: \$100,000 / 1000 QPS = \$100/QPS

벤치마크의 목적: 공정한 비교

- 재현 조건:
- 1. 하드웨어: 동일 모델 사용
 - 2. 소프트웨어: DB 버전, OS, 컴파일러 공개
 - 3. 데이터: 표준 데이터 생성 도구로 동일 데이터 생성
 - 4. 튜닝: 각 벤더가 최적 설정으로 조정 (공정함)
 - 5. 검증: 독립적 감시자가 감시

결과:
다른 벤더도 같은 환경에서 재테스트 가능
결과 신뢰성 높음

STEW q사생생 관계D

k

P관계hN5S

4

D

```
원본 TPC-H:
CREATE TABLE part (
  p_partkey INT PRIMARY KEY,
  p_name VARCHAR(55),
  p_mfgr VARCHAR(25),
  p_brand VARCHAR(10),
  ...
)
```

```
Exqutor 확장:
CREATE TABLE part (
  p_partkey INT PRIMARY KEY,
  p_name VARCHAR(55),
  p_mfgr VARCHAR(25),
  p_brand VARCHAR(10),
  ...,
  p_embedding FLOAT8[] -- * 추가됨
)
```

D

- 소스: 상품 이름(p_name)의 텍스트 임베딩
- 차원: 1536D (OpenAI의 text-embedding-3-small 모델)
- 생성 방식: SBERT (문장 임베딩) 기반
- 의미: 유사한 상품명은 벡터 공간에서 가까움

```
예:
"High-grade Brass Cable" → [0.1, 0.2, 0.5, ...]
"Premium Copper Wire" → [0.12, 0.21, 0.48, ...] (가까움)
"Plastic Bucket" → [0.9, 0.1, 0.05, ...] (멀음)
```

관계가 없는 hdN5S B kLe D

```
Q3 (배송 우선순위) → Q3-VAQ:
WHERE ... AND p_embedding <-> query_vec < 0.5
```

```
Q5 (지역별 매출) → Q5-VAQ:
WHERE ... AND p_embedding <-> query_vec < 0.5
```

```
Q8 (시장 점유율) → Q8-VAQ:
WHERE ... AND p_embedding <-> query_vec < 0.5
```

Q9, Q10, Q11, Q12, Q20: 유사하게 확장

Q: 85kLe 1D

```
-- 원본
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
  AND n.n_name = 'CANADA'
  AND s.s_suppkey IN (
    SELECT ps.ps_suppkey
    FROM partsupp ps
    WHERE ps.ps_partkey IN (
      SELECT p.p_partkey
      FROM part p
      WHERE p.p_type = 'ECONOMY' -- 단순 필터
    )
  )

-- Exqutor 확장
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
  AND n.n_name = 'CANADA'
  AND s.s_suppkey IN (
    SELECT ps.ps_suppkey
```

```
FROM partsupp ps
WHERE ps.ps_partkey IN (
  SELECT p.p_partkey
  FROM part p
  WHERE p.p_embedding <-> query_vec < 0.5 -- ★ 벡터 조건 추가
)
)
```

STP q사실생

과목별

SF (Scale Factor) = 테이블 크기
SF=1 (1GB, lineitem 600만 행): pgvector (기본): 0.5초 ~ 5초 pgvector + Exqutor: 0.05초 ~ 0.5초 (5~10배 향상)
SF=10 (10GB, lineitem 6,000만 행): pgvector (기본): 5초 ~ 50초 pgvector + Exqutor: 0.5초 ~ 5초 (10~20배 향상)
SF=100 (100GB, lineitem 6억 행): pgvector (기본): 50초 ~ 500초 pgvector + Exqutor: 5초 ~ 50초 (10~50배 향상)

Q3-VAQ (3개 테이블 조인): 향상도: 8.7배
Q5-VAQ (6개 테이블 조인): 향상도: 15.2배
Q8-VAQ (8개 테이블 조인, 서브쿼리 있음): 향상도: 48.9배 (★ 최대)
Q20-VAQ (복잡한 중첩 서브쿼리): 향상도: 37.5배

D4
원인: 카디널리티 오추정의 영향이 누적되기 때문
- 3개 테이블: 오추정 영향 작음
- 6개 테이블: 오추정이 조인 순서 결정에 큰 영향
- 8개 이상: 오추정으로 인한 계획 오류가 극단적

STP

hdNSSD
96
OM
P관계명
: 6
<6
e9e: 9
P관계명
P관계명D

다른 벤치마크도 있는데:
- TPCH: 사실상의 산업 표준
- TPCDS: 더 복잡 (나중에 추가 실험)
- SPECjbb: Java 벤치마크 (DB 아님)

- Yahoo Cloud Serving Benchmark: 클라우드 (쿼리 단순)

TPC-H 선택:

- 공신력 높음
- 학계와 업계 모두에서 사용
- 확장 가능 (임베딩 추가 용이)
- 결과 비교 가능

96

고객 1 D
hdN5S I
4 I
6

: 6

B D
e 94e : 4e = I
e ? 4e A4e C I
6

<6gyv개Qvy개만

hdN5S D

SF=100이 "큰 벤치마크"로 간주됨 (100GB)

최신 데이터 웨어하우스:

- 테라바이트 규모 (1000배)
- 페타바이트도 가능

SF=100 결과가 실제 프로덕션 환경을 대표하는가?

PE

P관계만 것 1개x- | 값 5개 1개 1개 공 y F 86 D
061 I
I

SE

c g 40M D
D Tc
D
I
0 1D
0 1D

TE

OM D
P관계만 I
공결